

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
<head>
<title>TestPage</title>
```

```
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.

2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I B. Maheshwar (192411195) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B. Maheshwar

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1. HTTP Response Analysis

1. Identify any incorrect or invalid fields in the header.

The following HTTP response header is given:

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

Incorrect field:

- Content-Length: -150 is invalid because the Content-Length value cannot be negative.
- All other fields are valid and correctly formatted.

2. Analyze the issue with Content-Length and explain its impact.

- The Content-Length header specifies the size of the response body in bytes.
- A negative value is not allowed by the HTTP specification.
- Because of this error, the browser cannot determine the correct size of the response.
- It may result in:
 - Failure to load the webpage.
 - Truncated or incomplete content.
 - Connection errors.
 - Browser ignoring the response.

3. Determine whether the status code matches the response correctness.

- 200 OK means the request was successfully processed.
- However, since the Content-Length value is invalid, the response contains an error.
- Therefore, the status code does not completely match the correctness of the response.
- The server should either send the correct Content-Length or return an appropriate error status if the response cannot be delivered correctly.

4. Suggest corrections for the identified errors.

Corrected HTTP header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

Corrections:

- Replace -150 with the actual response size (e.g., 150).
- Ensure all header values follow HTTP standards.

2. DOM Structure Analysis

Given DOM:

```
>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

1. Identify missing or improperly closed elements.

Missing closing tags:

- `</p>` is missing.
- `</div>` appears before closing the `<p>` element.

2. Analyze how the browser will interpret this structure.

- Modern browsers automatically correct invalid HTML.
- The browser will automatically insert the missing `</p>` tag before the `</div>`.
- The webpage will still be displayed correctly in most browsers.

3. Determine the impact on rendering and layout.

- The page may render correctly due to browser error correction.
- However, invalid HTML can:
 - Cause inconsistent rendering across browsers.
 - Create problems for CSS styling.
 - Affect JavaScript DOM manipulation.
 - Reduce maintainability.

4. Suggest corrections to make the DOM valid.

Correct HTML:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>Hello World</p>
</div>
</body>
</html>
```

3. GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

GET

- Sends data through the URL.
- Data is visible to everyone.
- Limited amount of data.
- Used for retrieving information.

POST

- Sends data inside the request body.
- Data is not visible in the URL.

- Supports larger amounts of data.
- Used for submitting sensitive information.

2. Analyze which method is more suitable for login and why.

POST is more suitable because:

- Username and password are hidden from the URL.
- Prevents credentials from being stored in browser history.
- Provides better security than GET.
- Commonly used for login and registration forms.

3. Identify security risks in using GET for authentication.

Using GET can:

- Expose passwords in the browser address bar.
- Store credentials in browser history.
- Save passwords in server logs.
- Allow credentials to be shared through copied URLs.
- Increase the risk of information leakage.

4. Suggest best practices for secure login implementation.

- Use **POST** instead of GET.
- Use **HTTPS** to encrypt communication.
- Hash passwords before storing them.
- Validate user input.
- Implement secure session management.
- Enable multi-factor authentication (MFA) where possible.

4. HTML Code Inspection

Given HTML:

```
!DOCTYPE html>
<html>
<head>
<title>Sample</title>
```



```
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

1. Identify missing closing tags.

Missing tags:

- `</h1>`
- `</p>`

2. Analyze issues with the `<img>` tag.

- The `<img>` tag is a void element and does not require a closing tag.
- However, it should include:
 - alt attribute
 - width and height (optional but recommended)

Example:

```

```

3. Determine how browsers will handle these errors.

- Browsers automatically close the missing tags.
- The webpage usually displays correctly.
- However:
 - HTML becomes invalid.
 - CSS layout may behave unexpectedly.
 - JavaScript may not work correctly.
 - Accessibility decreases.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html><html><head><title>Sample</title></head><body>
<h1>Welcome</h1>
<p>This is a paragraph.</p>

</body></html>
```

5. Browser–Server Communication Flow

1. Interpret each step in the communication flow.

Step 1: User enters a URL in the browser.

- The browser receives the website address.

Step 2: DNS lookup occurs.

- DNS converts the domain name into an IP address.

Step 3: HTTP request is sent.

- The browser sends a request to the web server.

Step 4: Server processes the request.

- The server retrieves the requested webpage or data.

Step 5: Response is returned.

- The server sends an HTTP response containing HTML, CSS, JavaScript, or other resources.

Step 6: Browser renders the page.

- The browser downloads the resources and displays the webpage to the user.

2. Identify where delays can occur.

Delays may occur during:

- DNS lookup.
- Slow internet connection.
- High network latency.
- Server processing time.

- Large webpage downloads.
- Rendering complex JavaScript and CSS.

3. Analyze the role of DNS in the process.

- DNS (Domain Name System) translates a domain name into its corresponding IP address.
- Without DNS, browsers cannot locate the correct web server.
- DNS acts like the Internet's phonebook, allowing users to access websites using easy-to-remember

4. Suggest optimizations to improve performance.

- Use DNS caching to reduce lookup time.
- Enable browser caching.
- Compress files using Gzip or Brotli.
- Use a Content Delivery Network (CDN).
- Minify HTML, CSS, and JavaScript files.
- Optimize images and multimedia.
- Reduce unnecessary HTTP requests.
- Use HTTPS with HTTP/2 or HTTP/3 for faster communication.